Systamatic Solvers

```
/* ==========================================================
   BIKE RENTAL DATA INSIGHTS HACKATHON
   FINAL CLEANED SOLUTION
   ========================================================== */


/* ==========================================================
   1) DATABASE / SCHEMA SETUP
   ========================================================== */

CREATE OR REPLACE DATABASE BIKE_RENTAL_DB;
USE DATABASE BIKE_RENTAL_DB;

CREATE OR REPLACE SCHEMA RAW;
CREATE OR REPLACE SCHEMA VALIDATED;
CREATE OR REPLACE SCHEMA CURATED;
CREATE OR REPLACE SCHEMA UTIL;

/* Use your warehouse if already available */
-- USE WAREHOUSE YOUR_WAREHOUSE_NAME;


/* ==========================================================
   2) FILE FORMAT
   ========================================================== */
```

```
CREATE OR REPLACE FILE FORMAT UTIL.CSV_FMT
TYPE = 'CSV'
SKIP_HEADER = 1
FIELD_OPTIONALLY_ENCLOSED_BY = '"'
NULL_IF = ('NULL', 'null', '');



/* ==========================================================
  3) AWS S3 STAGE
  Replace with your bucket and credentials
  ========================================================== */

CREATE OR REPLACE STAGE UTIL.BIKE_STAGE
URL = 's3://rajesh64/'
CREDENTIALS = (
   AWS_KEY_ID = 'AKIA6OZQ4B7OJN7TPRWD'
   AWS_SECRET_KEY = 'dEFFYT57QJ2czUAkjKdOek48E/nT6SljOKda9+9e'
)
FILE_FORMAT = UTIL.CSV_FMT;

/* Check files */
LIST @UTIL.BIKE_STAGE;



/* ==========================================================
  4) RAW TABLES
```

```
  RAW keeps landed source data
  ========================================================== */

CREATE OR REPLACE TABLE RAW.STATIONS_RAW (
  station_id       NUMBER,
  station_name     STRING,
  latitude         FLOAT,
  longitude        FLOAT,
  capacity         NUMBER,
  neighborhood     STRING,
  city_zone        STRING,
  install_date     DATE,
  status           STRING,
  source_file_name STRING,
  load_ts          TIMESTAMP_NTZ
);

CREATE OR REPLACE TABLE RAW.BIKES_RAW (
  bike_id            NUMBER,
  bike_type          STRING,
  status             STRING,
  purchase_date      DATE,
  last_service_date  DATE,
  odometer_km        NUMBER,
  battery_level      NUMBER,
  firmware_version   STRING,
  source_file_name   STRING,
```

```
    load_ts          TIMESTAMP_NTZ
);

CREATE OR REPLACE TABLE RAW.USERS_RAW (
    user_id           NUMBER,
    customer_name      STRING,
    dob               DATE,
    gender            STRING,
    email             STRING,
    phone             STRING,
    address           STRING,
    city              STRING,
    state             STRING,
    region            STRING,
    kyc_status        STRING,
    registration_date   DATE,
    is_student        NUMBER,
    corporate_id       STRING,
    source_file_name    STRING,
    load_ts           TIMESTAMP_NTZ
);

CREATE OR REPLACE TABLE RAW.RENTALS_RAW (
    rental_id         NUMBER,
    user_id           NUMBER,
    bike_id           NUMBER,
    start_station_id   NUMBER,
```

```
    end_station_id     NUMBER,
    start_time         DATE,
    end_time           DATE,
    duration_sec       NUMBER,
    distance_km        FLOAT,
    price              FLOAT,
    plan_type          STRING,
    channel            STRING,
    device_info        STRING,
    start_gps          STRING,
    end_gps            STRING,
    is_flagged         NUMBER,
    source_file_name   STRING,
    load_ts            TIMESTAMP_NTZ
);
select *from rentals_raw;

/* ========================================================
   5) LOAD MASTER FILES INTO RAW
   ======================================================== */

COPY INTO RAW.STATIONS_RAW
FROM (
    SELECT
        $1::NUMBER,
        $2::STRING,
        $3::FLOAT,
```

```
        $4::FLOAT,
        $5::NUMBER,
        $6::STRING,
        $7::STRING,
        $8::DATE,
        $9::STRING,
        METADATA$FILENAME,
        CURRENT_TIMESTAMP()
    FROM @UTIL.BIKE_STAGE/stations_master.csv
)
ON_ERROR = 'CONTINUE';

COPY INTO RAW.BIKES_RAW
FROM (
    SELECT
        $1::NUMBER,
        $2::STRING,
        $3::STRING,
        $4::DATE,
        $5::DATE,
        $6::NUMBER,
        $7::NUMBER,
        $8::STRING,
        METADATA$FILENAME,
        CURRENT_TIMESTAMP()
    FROM @UTIL.BIKE_STAGE/bikes_master.csv
)
```

```
ON_ERROR = 'CONTINUE';


select *from bikes_raw;

select bike_type  from  bikes_raw group by bike_type ;


select r.rental_id,u.user_id , u.customer_name    from rentals_raw as r join   users_raw as u where r.rental_id = u.user_id ;


select *from rentals_raw;


SELECT COUNT(CASE WHEN price > 150 then 1 end ) AS high, COUNT(CASE WHEN price < 60 then 1 end) AS low FROM rentals_raw;




COPY INTO RAW.USERS_RAW

FROM (

   SELECT

      $1::NUMBER,

      $2::STRING,

      $3::DATE,

      $4::STRING,

      $5::STRING,

      $6::STRING,

      $7::STRING,

      $8::STRING,

      $9::STRING,

      $10::STRING,
```

```
        $11::STRING,
        $12::DATE,
        $13::NUMBER,
        $14::STRING,
        METADATA$FILENAME,
        CURRENT_TIMESTAMP()
    FROM @UTIL.BIKE_STAGE/users_master.csv
)
ON_ERROR = 'CONTINUE';


COPY INTO RAW.RENTALS_RAW
FROM (
    SELECT
        $1::NUMBER,
        $2::NUMBER,
        $3::NUMBER,
        $4::NUMBER,
        $5::NUMBER,
        $6::DATE,
        $7::DATE,
        $8::NUMBER,
        $9::FLOAT,
        $10::FLOAT,
        $11::STRING,
        $12::STRING,
        $13::STRING,
        $14::STRING,
```

```
      $15::STRING,
      $16::NUMBER,
      METADATA$FILENAME,
      CURRENT_TIMESTAMP()
   FROM @UTIL.BIKE_STAGE/rentals_master.csv
)
ON_ERROR = 'CONTINUE';



/* ==========================================================
  6) DQ EXCEPTIONS TABLE
  Stores invalid rejected rows
  ========================================================== */

CREATE OR REPLACE TABLE VALIDATED.DQ_EXCEPTIONS (
   table_name       STRING,
   record_id        STRING,
   issue_description STRING,
   source_file_name  STRING,
   load_ts          TIMESTAMP_NTZ,
   logged_ts        TIMESTAMP_NTZ DEFAULT CURRENT_TIMESTAMP()
);

/* Log invalid master rows */

INSERT INTO VALIDATED.DQ_EXCEPTIONS
SELECT
```

```
    'RENTALS',
    rental_id::STRING,
    'Invalid duration or distance or end_time before start_time',
    source_file_name,
    load_ts,
    CURRENT_TIMESTAMP()
FROM RAW.RENTALS_RAW
WHERE duration_sec < 0
   OR distance_km < 0
   OR end_time < start_time;

INSERT INTO VALIDATED.DQ_EXCEPTIONS
SELECT
    'BIKES',
    bike_id::STRING,
    'Invalid odometer or battery level',
    source_file_name,
    load_ts,
    CURRENT_TIMESTAMP()
FROM RAW.BIKES_RAW
WHERE odometer_km < 0
   OR battery_level NOT BETWEEN 0 AND 100;

INSERT INTO VALIDATED.DQ_EXCEPTIONS
SELECT
    'STATIONS',
    station_id::STRING,
```

```
    'Invalid capacity or missing coordinates',
    source_file_name,
    load_ts,
    CURRENT_TIMESTAMP()
FROM RAW.STATIONS_RAW
WHERE capacity < 0
   OR latitude IS NULL
   OR longitude IS NULL;

INSERT INTO VALIDATED.DQ_EXCEPTIONS
SELECT
    'USERS',
    COALESCE(user_id::STRING, 'NULL'),
    'Missing user_id',
    source_file_name,
    load_ts,
    CURRENT_TIMESTAMP()
FROM RAW.USERS_RAW
WHERE user_id IS NULL;


/* ==========================================================
   7) VALIDATED LAYER
   Clean + deduplicate
   Latest loaded row wins
   ========================================================== */
```

```
CREATE OR REPLACE TABLE VALIDATED.STATIONS_V AS
SELECT
  station_id,
  station_name,
  latitude,
  longitude,
  capacity,
  neighborhood,
  city_zone,
  install_date,
  LOWER(status) AS status,
  source_file_name,
  load_ts
FROM (
  SELECT *,
      ROW_NUMBER() OVER (PARTITION BY station_id ORDER BY load_ts DESC) AS rn
  FROM RAW.STATIONS_RAW
  WHERE station_id IS NOT NULL
    AND capacity >= 0
    AND latitude IS NOT NULL
    AND longitude IS NOT NULL
)
WHERE rn = 1;

CREATE OR REPLACE TABLE VALIDATED.BIKES_V AS
SELECT
  bike_id,
```

```
    LOWER(bike_type) AS bike_type,
    LOWER(status) AS status,
    purchase_date,
    last_service_date,
    odometer_km,
    battery_level,
    firmware_version,
    source_file_name,
    load_ts
FROM (
    SELECT *,
        ROW_NUMBER() OVER (PARTITION BY bike_id ORDER BY load_ts DESC) AS rn
    FROM RAW.BIKES_RAW
    WHERE bike_id IS NOT NULL
      AND odometer_km >= 0
      AND battery_level BETWEEN 0 AND 100
)
WHERE rn = 1;

CREATE OR REPLACE TABLE VALIDATED.USERS_V AS
SELECT
    user_id,
    customer_name,
    dob,
    UPPER(gender) AS gender,
    LOWER(email) AS email,
    phone,
```

```
    address,
    city,
    state,
    region,
    LOWER(kyc_status) AS kyc_status,
    registration_date,
    is_student,
    corporate_id,
    source_file_name,
    load_ts
FROM (
    SELECT *,
        ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY load_ts DESC) AS rn
    FROM RAW.USERS_RAW
    WHERE user_id IS NOT NULL
)
WHERE rn = 1;

CREATE OR REPLACE TABLE VALIDATED.RENTALS_V AS
SELECT
    rental_id,
    user_id,
    bike_id,
    start_station_id,
    end_station_id,
    start_time,
    end_time,
```

```
    duration_sec,
    distance_km,
    price,
    LOWER(plan_type) AS plan_type,
    LOWER(channel) AS channel,
    device_info,
    start_gps,
    end_gps,
    is_flagged,
    source_file_name,
    load_ts
FROM (
    SELECT *,
           ROW_NUMBER() OVER (PARTITION BY rental_id ORDER BY load_ts DESC) AS rn
    FROM RAW.RENTALS_RAW
    WHERE rental_id IS NOT NULL
      AND duration_sec >= 0
      AND distance_km >= 0
      AND end_time >= start_time
)
WHERE rn = 1;




/* ==========================================================
   9) CURATED TABLES
```

```
  Dimensions use SCD Type 2
  Fact stores rental transactions
  ========================================================== */

CREATE OR REPLACE TABLE CURATED.DIM_STATIONS_SCD2 (
  station_sk       NUMBER AUTOINCREMENT,
  station_id       NUMBER,
  station_name     STRING,
  latitude         FLOAT,
  longitude        FLOAT,
  capacity         NUMBER,
  neighborhood     STRING,
  city_zone        STRING,
  install_date     DATE,
  status           STRING,
  effective_from   TIMESTAMP_NTZ,
  effective_to     TIMESTAMP_NTZ,
  is_current       BOOLEAN
);

CREATE OR REPLACE TABLE CURATED.DIM_BIKES_SCD2 (
  bike_sk          NUMBER AUTOINCREMENT,
  bike_id          NUMBER,
  bike_type        STRING,
  status           STRING,
  purchase_date    DATE,
  last_service_date  DATE,
```

```
    odometer_km        NUMBER,
    battery_level      NUMBER,
    firmware_version   STRING,
    effective_from     TIMESTAMP_NTZ,
    effective_to       TIMESTAMP_NTZ,
    is_current         BOOLEAN
);

CREATE OR REPLACE TABLE CURATED.DIM_USERS_SCD2 (
    user_sk            NUMBER AUTOINCREMENT,
    user_id            NUMBER,
    customer_name      STRING,
    dob                DATE,
    gender             STRING,
    email              STRING,
    phone              STRING,
    address            STRING,
    city               STRING,
    state              STRING,
    region             STRING,
    kyc_status         STRING,
    registration_date  DATE,
    is_student         NUMBER,
    corporate_id       STRING,
    effective_from     TIMESTAMP_NTZ,
    effective_to       TIMESTAMP_NTZ,
    is_current         BOOLEAN
```

```
);

CREATE OR REPLACE TABLE CURATED.FACT_RENTALS (
    rental_id                NUMBER,
    user_id                  NUMBER,
    bike_id                  NUMBER,
    start_station_id         NUMBER,
    end_station_id           NUMBER,
    start_time               DATE,
    end_time                 DATE,
    duration_sec             NUMBER,
    distance_km              FLOAT,
    price                    FLOAT,
    plan_type                STRING,
    channel                  STRING,
    device_info              STRING,
    start_gps                STRING,
    end_gps                  STRING,
    is_flagged               NUMBER,
    rental_day               DATE,
    rental_week              NUMBER,
    anomaly_short_trip       NUMBER,
    anomaly_gps_mismatch     NUMBER,
    anomaly_zero_battery     NUMBER,
    anomaly_device_reuse     NUMBER,
    anomaly_station_geofence NUMBER,
    anomaly_score            NUMBER,
```

```
    inserted_ts              TIMESTAMP_NTZ DEFAULT CURRENT_TIMESTAMP()
);




/* ==========================================================
   11) LOAD INCREMENT FILES INTO RAW
   ========================================================== */


COPY INTO RAW.STATIONS_RAW
FROM (
    SELECT
        $1::NUMBER,
        $2::STRING,
        $3::FLOAT,
        $4::FLOAT,
        $5::NUMBER,
        $6::STRING,
        $7::STRING,
        $8::DATE,
        $9::STRING,
        METADATA$FILENAME,
        CURRENT_TIMESTAMP()
    FROM @UTIL.BIKE_STAGE/stations_inc.csv
)
ON_ERROR = 'CONTINUE';
```

```
COPY INTO RAW.BIKES_RAW
FROM (
   SELECT
      $1::NUMBER,
      $2::STRING,
      $3::STRING,
      $4::DATE,
      $5::DATE,
      $6::NUMBER,
      $7::NUMBER,
      $8::STRING,
      METADATA$FILENAME,
      CURRENT_TIMESTAMP()
   FROM @UTIL.BIKE_STAGE/bikes_inc.csv
)
ON_ERROR = 'CONTINUE';

COPY INTO RAW.USERS_RAW
FROM (
   SELECT
      $1::NUMBER,
      $2::STRING,
      $3::DATE,
      $4::STRING,
      $5::STRING,
      $6::STRING,
```

```
        $7::STRING,
        $8::STRING,
        $9::STRING,
        $10::STRING,
        $11::STRING,
        $12::DATE,
        $13::NUMBER,
        $14::STRING,
        METADATA$FILENAME,
        CURRENT_TIMESTAMP()
    FROM @UTIL.BIKE_STAGE/users_inc.csv
)
ON_ERROR = 'CONTINUE';

COPY INTO RAW.RENTALS_RAW
FROM (
    SELECT
        $1::NUMBER,
        $2::NUMBER,
        $3::NUMBER,
        $4::NUMBER,
        $5::NUMBER,
        $6::DATE,
        $7::DATE,
        $8::NUMBER,
        $9::FLOAT,
        $10::FLOAT,
```

```
        $11::STRING,
        $12::STRING,
        $13::STRING,
        $14::STRING,
        $15::STRING,
        $16::NUMBER,
        METADATA$FILENAME,
        CURRENT_TIMESTAMP()
    FROM @UTIL.BIKE_STAGE/rentals_inc.csv
)
ON_ERROR = 'CONTINUE';

/* Log invalid increment rows */

INSERT INTO VALIDATED.DQ_EXCEPTIONS
SELECT
    'RENTALS',
    rental_id::STRING,
    'Invalid duration or distance or end_time before start_time',
    source_file_name,
    load_ts,
    CURRENT_TIMESTAMP()
FROM RAW.RENTALS_RAW
WHERE duration_sec < 0
   OR distance_km < 0
   OR end_time < start_time;
```

```
INSERT INTO VALIDATED.DQ_EXCEPTIONS
SELECT
    'BIKES',
    bike_id::STRING,
    'Invalid odometer or battery level',
    source_file_name,
    load_ts,
    CURRENT_TIMESTAMP()
FROM RAW.BIKES_RAW
WHERE odometer_km < 0
   OR battery_level NOT BETWEEN 0 AND 100;

INSERT INTO VALIDATED.DQ_EXCEPTIONS
SELECT
    'STATIONS',
    station_id::STRING,
    'Invalid capacity or missing coordinates',
    source_file_name,
    load_ts,
    CURRENT_TIMESTAMP()
FROM RAW.STATIONS_RAW
WHERE capacity < 0
   OR latitude IS NULL
   OR longitude IS NULL;

INSERT INTO VALIDATED.DQ_EXCEPTIONS
SELECT
```

```
    'USERS',
    COALESCE(user_id::STRING, 'NULL'),
    'Missing user_id',
    source_file_name,
    load_ts,
    CURRENT_TIMESTAMP()
FROM RAW.USERS_RAW
WHERE user_id IS NULL;



/* ========================================================
   12) RE-VALIDATE AFTER INCREMENT
   ======================================================== */

CREATE OR REPLACE TABLE VALIDATED.STATIONS_V AS
SELECT
    station_id,
    station_name,
    latitude,
    longitude,
    capacity,
    neighborhood,
    city_zone,
    install_date,
    LOWER(status) AS status,
    source_file_name,
    load_ts
```

```
FROM (
  SELECT *, ROW_NUMBER() OVER (PARTITION BY station_id ORDER BY load_ts DESC) AS rn
  FROM RAW.STATIONS_RAW
  WHERE station_id IS NOT NULL
    AND capacity >= 0
    AND latitude IS NOT NULL
    AND longitude IS NOT NULL
)
WHERE rn = 1;


CREATE OR REPLACE TABLE VALIDATED.BIKES_V AS
SELECT
  bike_id,
  LOWER(bike_type) AS bike_type,
  LOWER(status) AS status,
  purchase_date,
  last_service_date,
  odometer_km,
  battery_level,
  firmware_version,
  source_file_name,
  load_ts
FROM (
  SELECT *, ROW_NUMBER() OVER (PARTITION BY bike_id ORDER BY load_ts DESC) AS rn
  FROM RAW.BIKES_RAW
  WHERE bike_id IS NOT NULL
    AND odometer_km >= 0
```

```
    AND battery_level BETWEEN 0 AND 100
)
WHERE rn = 1;

CREATE OR REPLACE TABLE VALIDATED.USERS_V AS
SELECT
  user_id,
  customer_name,
  dob,
  UPPER(gender) AS gender,
  LOWER(email) AS email,
  phone,
  address,
  city,
  state,
  region,
  LOWER(kyc_status) AS kyc_status,
  registration_date,
  is_student,
  corporate_id,
  source_file_name,
  load_ts
FROM (
  SELECT *, ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY load_ts DESC) AS rn
  FROM RAW.USERS_RAW
  WHERE user_id IS NOT NULL
)
```

```
WHERE rn = 1;


CREATE OR REPLACE TABLE VALIDATED.RENTALS_V AS
SELECT
  rental_id,
  user_id,
  bike_id,
  start_station_id,
  end_station_id,
  start_time,
  end_time,
  duration_sec,
  distance_km,
  price,
  LOWER(plan_type) AS plan_type,
  LOWER(channel) AS channel,
  device_info,
  start_gps,
  end_gps,
  is_flagged,
  source_file_name,
  load_ts
FROM (
  SELECT *, ROW_NUMBER() OVER (PARTITION BY rental_id ORDER BY load_ts DESC) AS rn
  FROM RAW.RENTALS_RAW
  WHERE rental_id IS NOT NULL
    AND duration_sec >= 0
```

```
    AND distance_km >= 0
    AND end_time >= start_time
)
WHERE rn = 1;



/* ==========================================================
  13) SCD TYPE 2 MERGE FOR DIMENSIONS
  ========================================================== */


/* STATIONS */
UPDATE CURATED.DIM_STATIONS_SCD2 d
SET effective_to = CURRENT_TIMESTAMP(),
   is_current = FALSE
FROM VALIDATED.STATIONS_V s
WHERE d.station_id = s.station_id
  AND d.is_current = TRUE
  AND (
    NVL(d.station_name, '') <> NVL(s.station_name, '')
    OR NVL(d.latitude, -99999) <> NVL(s.latitude, -99999)
    OR NVL(d.longitude, -99999) <> NVL(s.longitude, -99999)
    OR NVL(d.capacity, -1) <> NVL(s.capacity, -1)
    OR NVL(d.neighborhood, '') <> NVL(s.neighborhood, '')
    OR NVL(d.city_zone, '') <> NVL(s.city_zone, '')
    OR NVL(d.install_date, '1900-01-01'::DATE) <> NVL(s.install_date, '1900-01-01'::DATE)
    OR NVL(d.status, '') <> NVL(s.status, '')
  );
```

```
INSERT INTO CURATED.DIM_STATIONS_SCD2
(
   station_id, station_name, latitude, longitude, capacity,
   neighborhood, city_zone, install_date, status,
   effective_from, effective_to, is_current
)
SELECT
   s.station_id, s.station_name, s.latitude, s.longitude, s.capacity,
   s.neighborhood, s.city_zone, s.install_date, s.status,
   CURRENT_TIMESTAMP(), NULL, TRUE
FROM VALIDATED.STATIONS_V s
LEFT JOIN CURATED.DIM_STATIONS_SCD2 d
  ON s.station_id = d.station_id
 AND d.is_current = TRUE
WHERE d.station_id IS NULL
  OR (
    NVL(d.station_name, '') <> NVL(s.station_name, '')
    OR NVL(d.latitude, -99999) <> NVL(s.latitude, -99999)
    OR NVL(d.longitude, -99999) <> NVL(s.longitude, -99999)
    OR NVL(d.capacity, -1) <> NVL(s.capacity, -1)
    OR NVL(d.neighborhood, '') <> NVL(s.neighborhood, '')
    OR NVL(d.city_zone, '') <> NVL(s.city_zone, '')
    OR NVL(d.install_date, '1900-01-01'::DATE) <> NVL(s.install_date, '1900-01-01'::DATE)
    OR NVL(d.status, '') <> NVL(s.status, '')
  );
```

```
select *from DIM_STATIONS_SCD2;

/* BIKES */

UPDATE CURATED.DIM_BIKES_SCD2 d

SET effective_to = CURRENT_TIMESTAMP(),

   is_current = FALSE

FROM VALIDATED.BIKES_V s

WHERE d.bike_id = s.bike_id

  AND d.is_current = TRUE

  AND (

     NVL(d.bike_type, '') <> NVL(s.bike_type, '')

     OR NVL(d.status, '') <> NVL(s.status, '')

     OR NVL(d.purchase_date, '1900-01-01'::DATE) <> NVL(s.purchase_date, '1900-01-01'::DATE)

     OR NVL(d.last_service_date, '1900-01-01'::DATE) <> NVL(s.last_service_date, '1900-01-01'::DATE)

     OR NVL(d.odometer_km, -1) <> NVL(s.odometer_km, -1)

     OR NVL(d.battery_level, -1) <> NVL(s.battery_level, -1)

     OR NVL(d.firmware_version, '') <> NVL(s.firmware_version, '')

  );


INSERT INTO CURATED.DIM_BIKES_SCD2

(

   bike_id, bike_type, status, purchase_date, last_service_date,

   odometer_km, battery_level, firmware_version,

   effective_from, effective_to, is_current

)

SELECT

   s.bike_id, s.bike_type, s.status, s.purchase_date, s.last_service_date,

   s.odometer_km, s.battery_level, s.firmware_version,
```

```
    CURRENT_TIMESTAMP(), NULL, TRUE
FROM VALIDATED.BIKES_V s
LEFT JOIN CURATED.DIM_BIKES_SCD2 d
  ON s.bike_id = d.bike_id
 AND d.is_current = TRUE
WHERE d.bike_id IS NULL
   OR (
      NVL(d.bike_type, '') <> NVL(s.bike_type, '')
      OR NVL(d.status, '') <> NVL(s.status, '')
      OR NVL(d.purchase_date, '1900-01-01'::DATE) <> NVL(s.purchase_date, '1900-01-01'::DATE)
      OR NVL(d.last_service_date, '1900-01-01'::DATE) <> NVL(s.last_service_date, '1900-01-01'::DATE)
      OR NVL(d.odometer_km, -1) <> NVL(s.odometer_km, -1)
      OR NVL(d.battery_level, -1) <> NVL(s.battery_level, -1)
      OR NVL(d.firmware_version, '') <> NVL(s.firmware_version, '')
   );

/* USERS */
UPDATE CURATED.DIM_USERS_SCD2 d
SET effective_to = CURRENT_TIMESTAMP(),
    is_current = FALSE
FROM VALIDATED.USERS_V s
WHERE d.user_id = s.user_id
  AND d.is_current = TRUE
  AND (
      NVL(d.customer_name, '') <> NVL(s.customer_name, '')
      OR NVL(d.dob, '1900-01-01'::DATE) <> NVL(s.dob, '1900-01-01'::DATE)
      OR NVL(d.gender, '') <> NVL(s.gender, '')
```

```
    OR NVL(d.email, '') <> NVL(s.email, '')
    OR NVL(d.phone, '') <> NVL(s.phone, '')
    OR NVL(d.address, '') <> NVL(s.address, '')
    OR NVL(d.city, '') <> NVL(s.city, '')
    OR NVL(d.state, '') <> NVL(s.state, '')
    OR NVL(d.region, '') <> NVL(s.region, '')
    OR NVL(d.kyc_status, '') <> NVL(s.kyc_status, '')
    OR NVL(d.registration_date, '1900-01-01'::DATE) <> NVL(s.registration_date, '1900-01-01'::DATE)
    OR NVL(d.is_student, -1) <> NVL(s.is_student, -1)
    OR NVL(d.corporate_id, '') <> NVL(s.corporate_id, '')
  );

INSERT INTO CURATED.DIM_USERS_SCD2
(
   user_id, customer_name, dob, gender, email, phone, address,
   city, state, region, kyc_status, registration_date,
   is_student, corporate_id, effective_from, effective_to, is_current
)
SELECT
   s.user_id, s.customer_name, s.dob, s.gender, s.email, s.phone, s.address,
   s.city, s.state, s.region, s.kyc_status, s.registration_date,
   s.is_student, s.corporate_id,
   CURRENT_TIMESTAMP(), NULL, TRUE
FROM VALIDATED.USERS_V s
LEFT JOIN CURATED.DIM_USERS_SCD2 d
  ON s.user_id = d.user_id
 AND d.is_current = TRUE
```

```
WHERE d.user_id IS NULL
  OR (
    NVL(d.customer_name, '') <> NVL(s.customer_name, '')
    OR NVL(d.dob, '1900-01-01'::DATE) <> NVL(s.dob, '1900-01-01'::DATE)
    OR NVL(d.gender, '') <> NVL(s.gender, '')
    OR NVL(d.email, '') <> NVL(s.email, '')
    OR NVL(d.phone, '') <> NVL(s.phone, '')
    OR NVL(d.address, '') <> NVL(s.address, '')
    OR NVL(d.city, '') <> NVL(s.city, '')
    OR NVL(d.state, '') <> NVL(s.state, '')
    OR NVL(d.region, '') <> NVL(s.region, '')
    OR NVL(d.kyc_status, '') <> NVL(s.kyc_status, '')
    OR NVL(d.registration_date, '1900-01-01'::DATE) <> NVL(s.registration_date, '1900-01-01'::DATE)
    OR NVL(d.is_student, -1) <> NVL(s.is_student, -1)
    OR NVL(d.corporate_id, '') <> NVL(s.corporate_id, '')
  );



/* ==========================================================
  14) FACT MERGE
  Insert only new rental rows
  ========================================================== */


MERGE INTO CURATED.FACT_RENTALS t
USING (
   WITH DEVICE_REUSE AS (
      SELECT device_info, COUNT(DISTINCT user_id) AS distinct_users
```

```
    FROM VALIDATED.RENTALS_V
    GROUP BY device_info
),
CURRENT_BIKES AS (
    SELECT bike_id, battery_level
    FROM CURATED.DIM_BIKES_SCD2
    WHERE is_current = TRUE
)
SELECT
    r.rental_id,
    r.user_id,
    r.bike_id,
    r.start_station_id,
    r.end_station_id,
    r.start_time,
    r.end_time,
    r.duration_sec,
    r.distance_km,
    r.price,
    r.plan_type,
    r.channel,
    r.device_info,
    r.start_gps,
    r.end_gps,
    r.is_flagged,
    r.start_time AS rental_day,
    WEEK(r.start_time) AS rental_week,
```

```
    CASE WHEN r.duration_sec <= 120 AND r.distance_km <= 0.5 THEN 1 ELSE 0 END AS anomaly_short_trip,
    CASE WHEN r.start_station_id = r.end_station_id AND r.distance_km > 10 THEN 1 ELSE 0 END AS anomaly_gps_mismatch,
    CASE WHEN b.battery_level = 0 THEN 1 ELSE 0 END AS anomaly_zero_battery,
    CASE WHEN d.distinct_users >= 3 THEN 1 ELSE 0 END AS anomaly_device_reuse,
    CASE WHEN r.start_station_id = r.end_station_id AND r.distance_km > 15 THEN 1 ELSE 0 END AS anomaly_station_geofence,
    (
      CASE WHEN r.duration_sec <= 120 AND r.distance_km <= 0.5 THEN 1 ELSE 0 END +
      CASE WHEN r.start_station_id = r.end_station_id AND r.distance_km > 10 THEN 1 ELSE 0 END +
      CASE WHEN b.battery_level = 0 THEN 1 ELSE 0 END +
      CASE WHEN d.distinct_users >= 3 THEN 1 ELSE 0 END +
      CASE WHEN r.start_station_id = r.end_station_id AND r.distance_km > 15 THEN 1 ELSE 0 END
    ) AS anomaly_score
  FROM VALIDATED.RENTALS_V r
  LEFT JOIN DEVICE_REUSE d ON r.device_info = d.device_info
  LEFT JOIN CURRENT_BIKES b ON r.bike_id = b.bike_id
) s
ON t.rental_id = s.rental_id
WHEN NOT MATCHED THEN
INSERT (
  rental_id, user_id, bike_id, start_station_id, end_station_id,
  start_time, end_time, duration_sec, distance_km, price,
  plan_type, channel, device_info, start_gps, end_gps, is_flagged,
  rental_day, rental_week,
  anomaly_short_trip, anomaly_gps_mismatch, anomaly_zero_battery,
  anomaly_device_reuse, anomaly_station_geofence, anomaly_score
```

```
)
VALUES (
    s.rental_id, s.user_id, s.bike_id, s.start_station_id, s.end_station_id,
    s.start_time, s.end_time, s.duration_sec, s.distance_km, s.price,
    s.plan_type, s.channel, s.device_info, s.start_gps, s.end_gps, s.is_flagged,
    s.rental_day, s.rental_week,
    s.anomaly_short_trip, s.anomaly_gps_mismatch, s.anomaly_zero_battery,
    s.anomaly_device_reuse, s.anomaly_station_geofence, s.anomaly_score
);



/* ========================================================
   15) AUTOMATION
   Procedure + stream + task
   ======================================================== */

CREATE OR REPLACE PROCEDURE UTIL.SP_REFRESH_VALIDATED()
RETURNS STRING
LANGUAGE SQL
AS
$$
BEGIN
    CREATE OR REPLACE TABLE VALIDATED.STATIONS_V AS
    SELECT
        station_id, station_name, latitude, longitude, capacity,
        neighborhood, city_zone, install_date, LOWER(status) AS status,
        source_file_name, load_ts
```

```
FROM (
   SELECT *, ROW_NUMBER() OVER (PARTITION BY station_id ORDER BY load_ts DESC) AS rn
   FROM RAW.STATIONS_RAW
   WHERE station_id IS NOT NULL
     AND capacity >= 0
     AND latitude IS NOT NULL
     AND longitude IS NOT NULL
)
WHERE rn = 1;

CREATE OR REPLACE TABLE VALIDATED.BIKES_V AS
SELECT
   bike_id, LOWER(bike_type) AS bike_type, LOWER(status) AS status,
   purchase_date, last_service_date, odometer_km, battery_level,
   firmware_version, source_file_name, load_ts
FROM (
   SELECT *, ROW_NUMBER() OVER (PARTITION BY bike_id ORDER BY load_ts DESC) AS rn
   FROM RAW.BIKES_RAW
   WHERE bike_id IS NOT NULL
     AND odometer_km >= 0
     AND battery_level BETWEEN 0 AND 100
)
WHERE rn = 1;

CREATE OR REPLACE TABLE VALIDATED.USERS_V AS
SELECT
   user_id, customer_name, dob, UPPER(gender) AS gender,
```

```
    LOWER(email) AS email, phone, address, city, state, region,
    LOWER(kyc_status) AS kyc_status, registration_date, is_student,
    corporate_id, source_file_name, load_ts
FROM (
    SELECT *, ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY load_ts DESC) AS rn
    FROM RAW.USERS_RAW
    WHERE user_id IS NOT NULL
)
WHERE rn = 1;


CREATE OR REPLACE TABLE VALIDATED.RENTALS_V AS
SELECT
    rental_id, user_id, bike_id, start_station_id, end_station_id,
    start_time, end_time, duration_sec, distance_km, price,
    LOWER(plan_type) AS plan_type, LOWER(channel) AS channel,
    device_info, start_gps, end_gps, is_flagged,
    source_file_name, load_ts
FROM (
    SELECT *, ROW_NUMBER() OVER (PARTITION BY rental_id ORDER BY load_ts DESC) AS rn
    FROM RAW.RENTALS_RAW
    WHERE rental_id IS NOT NULL
      AND duration_sec >= 0
      AND distance_km >= 0
      AND end_time >= start_time
)
WHERE rn = 1;
```

```
   RETURN 'Validated layer refreshed successfully';
END;
$$;

CREATE OR REPLACE STREAM UTIL.RENTALS_RAW_STREAM
ON TABLE RAW.RENTALS_RAW;

CREATE OR REPLACE TASK UTIL.TASK_REFRESH_VALIDATED
WAREHOUSE = COMPUTE_WH
SCHEDULE = 'USING CRON 0 * * * * UTC'
AS
CALL UTIL.SP_REFRESH_VALIDATED();

/* Start task when ready */
-- ALTER TASK UTIL.TASK_REFRESH_VALIDATED RESUME;


/* ==========================================================
  16) GOVERNANCE
  Dynamic masking for email and phone
  ========================================================== */

CREATE OR REPLACE MASKING POLICY CURATED.EMAIL_MASK AS
(val STRING) RETURNS STRING ->
CASE
   WHEN CURRENT_ROLE() IN ('ACCOUNTADMIN', 'SYSADMIN') THEN val
   ELSE '***MASKED_EMAIL***'
```

```
END;

CREATE OR REPLACE MASKING POLICY CURATED.PHONE_MASK AS
(val STRING) RETURNS STRING ->
CASE
   WHEN CURRENT_ROLE() IN ('ACCOUNTADMIN', 'SYSADMIN') THEN val
   ELSE '***MASKED_PHONE***'
END;

ALTER TABLE CURATED.DIM_USERS_SCD2
MODIFY COLUMN EMAIL
SET MASKING POLICY CURATED.EMAIL_MASK;

ALTER TABLE CURATED.DIM_USERS_SCD2
MODIFY COLUMN PHONE
SET MASKING POLICY CURATED.PHONE_MASK;


/* ==========================================================
  17) KPI VIEWS
  ========================================================== */

CREATE OR REPLACE VIEW CURATED.V_KPI_1_ANOMALOUS_RENTAL_PROBABILITY AS
SELECT
   ROUND(
      COUNT_IF(anomaly_score > 0) * 100.0 / NULLIF(COUNT(*), 0),
      2
```

```
    ) AS anomalous_rental_probability_score_pct
FROM CURATED.FACT_RENTALS;

CREATE OR REPLACE VIEW CURATED.V_KPI_2_STATION_AVAILABILITY_SCORE AS
SELECT
    start_station_id AS station_id,
    COUNT(*) AS total_rentals,
    ROUND(AVG(CASE WHEN anomaly_score = 0 THEN 1 ELSE 0 END) * 100, 2) AS station_availability_score_pct
FROM CURATED.FACT_RENTALS
GROUP BY start_station_id;

CREATE OR REPLACE VIEW CURATED.V_KPI_3_ACTIVE_RIDER_ENGAGEMENT_RATIO AS
WITH MAX_DT AS (
    SELECT MAX(start_time) AS max_start_date
    FROM CURATED.FACT_RENTALS
),
ACTIVE_USERS AS (
    SELECT COUNT(DISTINCT r.user_id) AS active_user_count
    FROM CURATED.FACT_RENTALS r
    CROSS JOIN MAX_DT m
    WHERE r.start_time >= DATEADD(DAY, -30, m.max_start_date)
),
TOTAL_USERS AS (
    SELECT COUNT(DISTINCT user_id) AS total_user_count
    FROM CURATED.DIM_USERS_SCD2
    WHERE is_current = TRUE
)
```

```
SELECT
    ROUND(
        a.active_user_count * 100.0 / NULLIF(t.total_user_count, 0),
        2
    ) AS active_rider_engagement_ratio_pct
FROM ACTIVE_USERS a
CROSS JOIN TOTAL_USERS t;

CREATE OR REPLACE VIEW CURATED.V_KPI_4_FLEET_MAINTENANCE_HEALTH_INDEX AS
WITH MAX_DT AS (
    SELECT MAX(last_service_date) AS max_service_date
    FROM CURATED.DIM_BIKES_SCD2
)
SELECT
    ROUND(
        COUNT_IF(
            status NOT IN ('retired', 'out_of_service')
            AND battery_level >= 25
            AND last_service_date >= DATEADD(DAY, -180, max_service_date)
        ) * 100.0 / NULLIF(COUNT(*), 0),
        2
    ) AS fleet_maintenance_health_index_pct
FROM CURATED.DIM_BIKES_SCD2, MAX_DT
WHERE is_current = TRUE;

CREATE OR REPLACE VIEW CURATED.V_KPI_5_AVERAGE_RENTAL_REVENUE_BY_CHANNEL AS
SELECT
```

```
   channel,
   ROUND(AVG(price), 2) AS avg_rental_revenue
FROM CURATED.FACT_RENTALS
GROUP BY channel;



/* ==========================================================
  18) DISPLAY QUERIES
  ========================================================== */

/* Raw tables */
SELECT * FROM RAW.STATIONS_RAW ORDER BY station_id, load_ts;
SELECT * FROM RAW.BIKES_RAW ORDER BY bike_id, load_ts;
SELECT * FROM RAW.USERS_RAW ORDER BY user_id, load_ts;
SELECT * FROM RAW.RENTALS_RAW ORDER BY rental_id, load_ts;

/* Validated tables */
SELECT * FROM VALIDATED.STATIONS_V ORDER BY station_id;
SELECT * FROM VALIDATED.BIKES_V ORDER BY bike_id;
SELECT * FROM VALIDATED.USERS_V ORDER BY user_id;
SELECT * FROM VALIDATED.RENTALS_V ORDER BY rental_id;

/* DQ table */
SELECT * FROM VALIDATED.DQ_EXCEPTIONS ORDER BY logged_ts DESC;
SELECT table_name, COUNT(*) AS rejected_rows
FROM VALIDATED.DQ_EXCEPTIONS
GROUP BY table_name
```

```
ORDER BY table_name;

/* Curated dimensions */
SELECT * FROM CURATED.DIM_STATIONS_SCD2 ORDER BY station_id, effective_from;
SELECT * FROM CURATED.DIM_BIKES_SCD2 ORDER BY bike_id, effective_from;
SELECT * FROM CURATED.DIM_USERS_SCD2 ORDER BY user_id, effective_from;

/* Current SCD2 rows only */
SELECT * FROM CURATED.DIM_STATIONS_SCD2 WHERE is_current = TRUE ORDER BY station_id;
SELECT * FROM CURATED.DIM_BIKES_SCD2 WHERE is_current = TRUE ORDER BY bike_id;
SELECT * FROM CURATED.DIM_USERS_SCD2 WHERE is_current = TRUE ORDER BY user_id;

/* Historical SCD2 rows only */
SELECT * FROM CURATED.DIM_STATIONS_SCD2 WHERE is_current = FALSE ORDER BY station_id, effective_from;
SELECT * FROM CURATED.DIM_BIKES_SCD2 WHERE is_current = FALSE ORDER BY bike_id, effective_from;
SELECT * FROM CURATED.DIM_USERS_SCD2 WHERE is_current = FALSE ORDER BY user_id, effective_from;

/* SCD2 validation checks */
SELECT station_id, COUNT(*) AS current_rows
FROM CURATED.DIM_STATIONS_SCD2
WHERE is_current = TRUE
GROUP BY station_id
HAVING COUNT(*) > 1;

SELECT bike_id, COUNT(*) AS current_rows
```

```
FROM CURATED.DIM_BIKES_SCD2
WHERE is_current = TRUE
GROUP BY bike_id
HAVING COUNT(*) > 1;

SELECT user_id, COUNT(*) AS current_rows
FROM CURATED.DIM_USERS_SCD2
WHERE is_current = TRUE
GROUP BY user_id
HAVING COUNT(*) > 1;

/* Fact table */
SELECT * FROM CURATED.FACT_RENTALS ORDER BY rental_id;

/* Anomaly display */
SELECT
  rental_id,
  user_id,
  bike_id,
  anomaly_short_trip,
  anomaly_gps_mismatch,
  anomaly_zero_battery,
  anomaly_device_reuse,
  anomaly_station_geofence,
  anomaly_score
FROM CURATED.FACT_RENTALS
ORDER BY rental_id;
```

```
/* Duplicate fact check */
SELECT rental_id, COUNT(*) AS cnt
FROM CURATED.FACT_RENTALS
GROUP BY rental_id
HAVING COUNT(*) > 1
ORDER BY rental_id;

/* Masked data display */
SELECT
   user_id,
   customer_name,
   email,
   phone,
   city,
   state,
   region,
   is_current
FROM CURATED.DIM_USERS_SCD2
ORDER BY user_id, effective_from;


/* ==========================================================
  19) FINAL KPI RESULT QUERIES
  ========================================================== */

SELECT COUNT(*) AS final_station_count FROM VALIDATED.STATIONS_V;
```

```
SELECT COUNT(*) AS final_bike_count FROM VALIDATED.BIKES_V;

SELECT COUNT(*) AS final_user_count FROM VALIDATED.USERS_V;

SELECT COUNT(*) AS final_rental_count FROM VALIDATED.RENTALS_V;


SELECT * FROM CURATED.V_KPI_1_ANOMALOUS_RENTAL_PROBABILITY;

SELECT * FROM CURATED.V_KPI_2_STATION_AVAILABILITY_SCORE ORDER BY station_id;

SELECT * FROM CURATED.V_KPI_3_ACTIVE_RIDER_ENGAGEMENT_RATIO;

SELECT * FROM CURATED.V_KPI_4_FLEET_MAINTENANCE_HEALTH_INDEX;

SELECT * FROM CURATED.V_KPI_5_AVERAGE_RENTAL_REVENUE_BY_CHANNEL ORDER BY avg_rental_revenue DESC;


--------- KPI --------


--anomalous rental probability score

SELECT

ROUND(

COUNT_IF(anomaly_score > 0) * 100.0 / COUNT(*), 2

) AS anomaly_pct

FROM CURATED.FACT_RENTALS;


CREATE OR REPLACE TABLE CURATED.KPI_ANOMALY_BREAKDOWN AS

SELECT

   COUNT(*) AS total_rentals,

   SUM(anomaly_short_trip) AS short_trip_count,

   SUM(anomaly_gps_mismatch) AS gps_issue_count,

   SUM(anomaly_zero_battery) AS zero_battery_count,

   SUM(anomaly_device_reuse) AS device_reuse_count,

   SUM(anomaly_station_geofence) AS geofence_issue_count
```

```
FROM CURATED.FACT_RENTALS;

select *from CURATED.KPI_ANOMALY_BREAKDOWN;

--station availability score

SELECT
start_station_id,
ROUND(AVG(CASE WHEN anomaly_score = 0 THEN 1 ELSE 0 END)*100,2) AS availability_score
FROM CURATED.FACT_RENTALS
GROUP BY start_station_id;

--active rider engagement ratio


WITH active_users AS (
SELECT COUNT(DISTINCT user_id) AS active
FROM CURATED.FACT_RENTALS
WHERE start_time >= DATEADD(DAY,-30,CURRENT_DATE)
),
total_users AS (
SELECT COUNT(DISTINCT user_id) AS total
FROM CURATED.DIM_USERS_SCD2
WHERE is_current = TRUE
)
SELECT ROUND(active*100.0/total,2) AS engagement_ratio
FROM active_users, total_users;
```

```
---fleet maintaince

SELECT
ROUND(
COUNT_IF(
battery_level >= 25
AND status = 'active'
) * 100.0 / COUNT(*),2
) AS health_index
FROM CURATED.DIM_BIKES_SCD2
WHERE is_current = TRUE;

-- average rental revenue by channel

SELECT channel, AVG(price)
FROM CURATED.FACT_RENTALS
GROUP BY channel;

--top performing stations
SELECT
start_station_id,
COUNT(*) AS total_rentals
FROM CURATED.FACT_RENTALS
GROUP BY start_station_id
ORDER BY total_rentals DESC;
```

```
--revenue per user

SELECT
user_id,
SUM(price) AS total_spent
FROM CURATED.FACT_RENTALS
GROUP BY user_id
ORDER BY total_spent DESC;

select * from users_raw;

SELECT
   email,
   SPLIT_PART(email, '@', 1) AS email_username
FROM RAW.USERS_RAW;


--All rides

CREATE OR REPLACE TABLE CURATED.ALL_RIDES AS
SELECT
   rental_id,
   user_id,
   bike_id,
   start_time,
   end_time,
   duration_sec,
```

```
    ROUND(duration_sec/60,2) AS duration_minutes,
    distance_km,
    price,
    channel,
    plan_type,
    start_station_id,
    end_station_id
FROM CURATED.FACT_RENTALS;



select *from all_rides;


--Top users


SELECT
start_station_id,
COUNT(*) AS total_rides
FROM CURATED.ALL_RIDES
GROUP BY start_station_id
ORDER BY total_rides DESC;


---bike info


CREATE OR REPLACE TABLE CURATED.BIKE_INFO AS
SELECT
    bike_id,
    bike_type,
```

```
    status,
    battery_level,
    odometer_km,
    purchase_date,
    last_service_date
FROM CURATED.DIM_BIKES_SCD2
WHERE is_current = TRUE;

select *from bike_info;
```